

ASSIGNMENT -2.5

NAME : JAWWAD KHAN

2303A51059

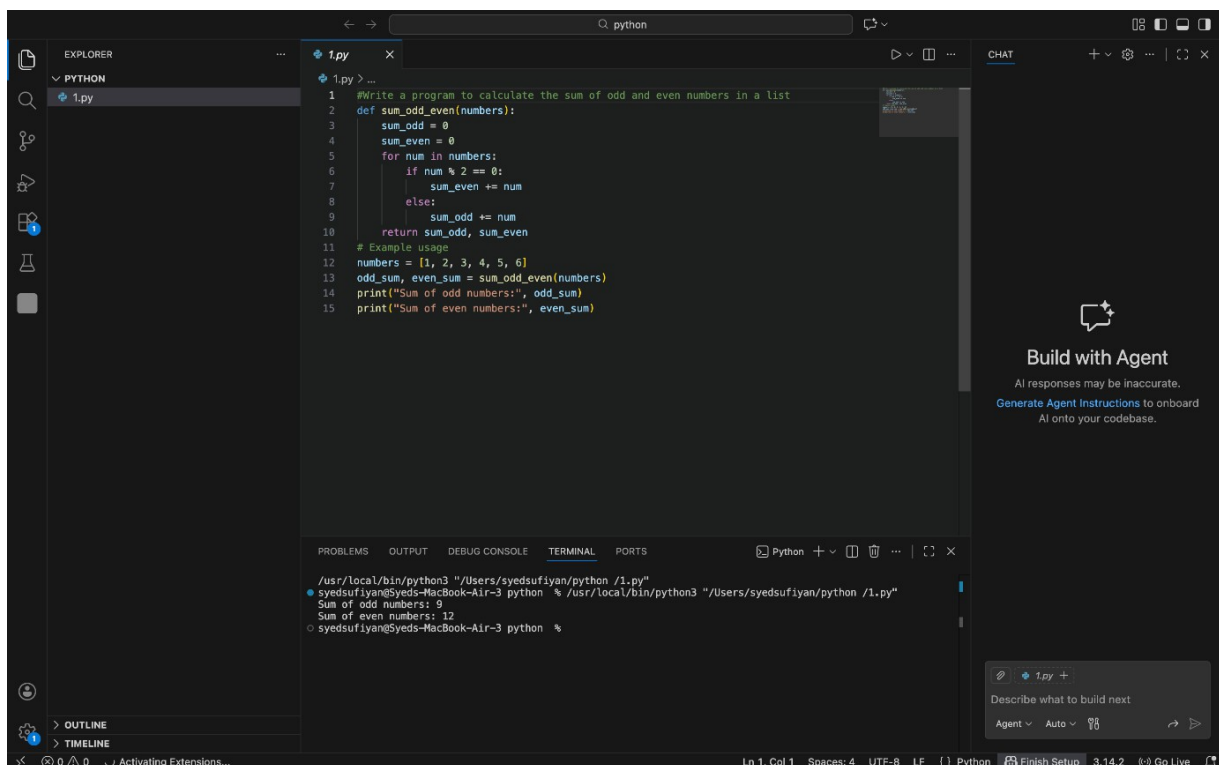
BATCH:30

TASK 1:

PROMPT:

WRITE A PROGRAM TO CALCULATE THE SUM OF ODD AND EVEN NUMBERS IN A LIST

CODE:



```
1.py > ...
1 #Write a program to calculate the sum of odd and even numbers in a list
2 def sum_odd_even(numbers):
3     sum_odd = 0
4     sum_even = 0
5     for num in numbers:
6         if num % 2 == 0:
7             sum_even += num
8         else:
9             sum_odd += num
10    return sum_odd, sum_even
11 # Example usage
12 numbers = [1, 2, 3, 4, 5, 6]
13 odd_sum, even_sum = sum_odd_even(numbers)
14 print("Sum of odd numbers:", odd_sum)
15 print("Sum of even numbers:", even_sum)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 "/Users/syedsufiyan/python/1.py"
syedsufiyan@syeds-MacBook-Air-3 python % /usr/local/bin/python3 "/Users/syedsufiyan/python/1.py"
Sum of odd numbers: 9
Sum of even numbers: 12
syedsufiyan@syeds-MacBook-Air-3 python %
```

Build with Agent

AI responses may be inaccurate.

[Generate Agent Instructions](#) to onboard AI onto your codebase.

Describe what to build next

Agent Auto

OBSERVATION:

The original code works correctly but is written as a single block, making it harder to reuse and test. The refactored (AI-improved) code separates logic into a function, improving:

- Readability
- Reusability
- Maintainability

Using a function allows the same logic to be reused with different lists without rewriting code.

TASK:2

PROMPT:

WRITE A PROGRAM EXPLAIN A FUNCTION THAT CALCULATES THE AREA OF DIFFERENT SHAPES. THE CODE MUST INCLUDE PROPER COMMENTS FOR EXPLANATION.

CODE:

```
1 # write a program explain a function that calculates the area of different shapes.
2 # The code must include proper comments for explanation.
3
4 def calculate_area(shape, value1, value2=0):
5     """
6     This function calculates the area of different shapes.
7     shape -> type of shape (circle, rectangle, triangle)
8     value1 -> radius (circle) OR length/base
9     value2 -> width/height (default is 0 for circle)
10    """
11
12    # Check if the shape is a circle
13    if shape == "circle":
14        return 3.14 * value1 * value1 #  $\pi * r * r$ 
15
16    # Check if the shape is a rectangle
17    elif shape == "rectangle":
18        return value1 * value2 # length * width
19
20    # Check if the shape is a triangle
21    elif shape == "triangle":
22        return 0.5 * value1 * value2 #  $\frac{1}{2} * \text{base} * \text{height}$ 
23
24    # If shape is not supported
25    else:
26        return "Invalid shape"
27
28
29 def main():
30     # Your code here
31
32     # Example usage:
33     # shape = input("Enter shape (circle/rectangle/triangle): ")
34     # radius = float(input("Enter radius/length/base: "))
35     # width = float(input("Enter width/height: "))
36     # area = calculate_area(shape, radius, width)
37     # print(f"Area of {shape}: {area}")
38
39     # Test cases
40     # print(calculate_area("circle", 5)) # 78.5
41     # print(calculate_area("rectangle", 10, 5)) # 50
42     # print(calculate_area("triangle", 10, 5)) # 25
43     # print(calculate_area("square", 5)) # Invalid shape
44
45     # Run the main function
46     main()
47
48 if __name__ == "__main__":
49     main()
```

Build with Agent
AI responses may be inaccurate.
[Generate Agent Instructions](#) to onboard AI onto your codebase.

Describe what to build next

Agent v Auto v

Ln 63, Col 7 Spaces: 4 UTF-8 LF Python Finish Setup 3.14.2 Go Live

OBSERVATION:

This program uses one function to calculate the area of multiple shapes, which avoids code duplication.

The shape parameter decides which formula to apply.

The function uses conditional statements (if /elif) to select the correct formula.

It improves code clarity, making onboarding easier and faster.

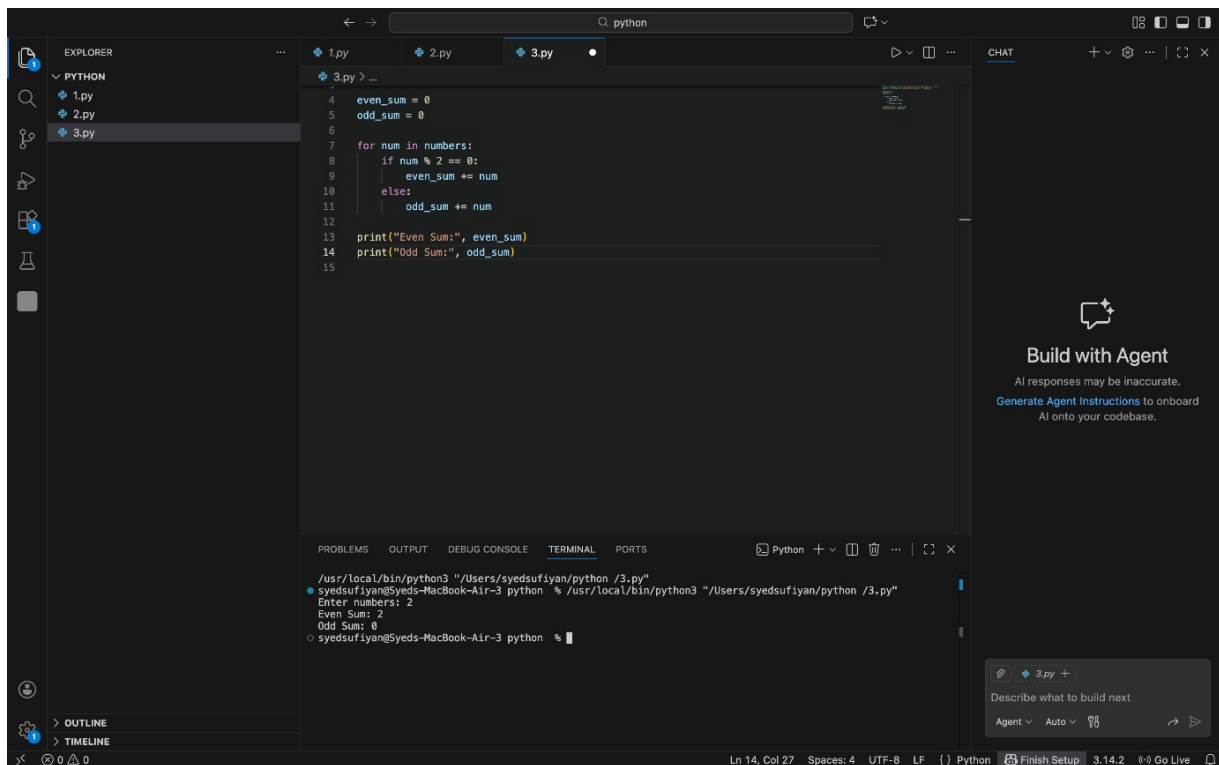
TASK:3

PROMPT:

EXPLAIN A FUNCTION THAT CALCULATES THE AREA OF DIFFERENT SHAPES (CURSER USED)

SHAPES. WRITE A PROGRAM TO FIND THE SUM OF EVEN AND ODD NUMBERS IN A LIST

CODE:



OBSERVATION:

The program demonstrates how one function can handle multiple use cases. Comments clearly explain:

What the function does

Why each condition exists

What each parameter represents

Using comments makes the code junior-developer friendly, which is ideal for onboarding.

The main () function separates user interaction from business logic, improving structure.

This style is considered clean, readable, and professional in real-world projects

TASK:4

PROMPT:

BASED ON PRACTICAL USAGE AND EXPERIMENTATION, COMPARE GEMINI, GITHUB COPILOT, AND CURSOR AI IN TERMS OF USABILITY AND CODE QUALITY. OBSERVATION:

GEMINI Is best suited for explanations and learning support. It produces readable, beginner-friendly code and clear step-by-step reasoning, making it ideal for onboarding juniors and understanding concepts.

GitHubT Copilo excels in real-time coding assistance inside IDEs. It is fast, context aware, and highly productive for experienced developers, but its code may lack explanations.

Cursor AI stands out for prompt sensitivity and refactoring quality. It responds strongly to detailed prompts, generating cleaner, more structured, and optimized code, making it suitable for improving legacy codebases.

usability, Copilot integrates seamlessly into workflows, Gemini is conversational and educational, and Cursor AI offers powerful prompt-driven refactoring.

code quality, Cursor AI and Copilot generally produce more professional, production ready code, while Gemini focuses on clarity over optimization

